Omega 61 (2016) 155–166

Contents lists available at ScienceDirect

Omega

journal homepage: www.elsevier.com/locate/omega

Pareto mimic algorithm: An approach to the team orienteering problem ☆

Liangjun Ke [a,*], Laipeng Zhai [a], Jing Li [b], Felix T.S. Chan [c]

[a] The State Key Laboratory for Manufacturing Systems Engineering, Xian Jiaotong University, Xi'an 710049, China
[b] The State Key Laboratory of Astronautic Dynamics Xi'an Satellite Control Center, Xi'an 710043, China
[c] Department of Industrial and Systems Engineering, The Hong Kong Polytechnic University, Hung Hom, Hong Kong

ARTICLE INFO

Article history:
Received 20 August 2013
Accepted 4 August 2015
Available online 12 August 2015

Keywords:
Vehicle scheduling
Vehicle routing problem
Team orienteering problem
Pareto dominance

ABSTRACT

The team orienteering problem is an important variant of the vehicle routing problem. In this paper, a new algorithm, called Pareto mimic algorithm, is proposed to deal with it. This algorithm maintains a population of incumbent solutions which are updated using Pareto dominance. It uses a new operator, called mimic operator, to generate a new solution by imitating an incumbent solution. Furthermore, to improve the quality of a solution, it employs an operator, called swallow operator which attempts to swallow (or insert) an infeasible node and then repair the resulting infeasible solution. A comparative study supports the effectiveness of the proposed algorithm, especially, our algorithm can quickly find new better results for several large-scale instances. We also demonstrate that Pareto mimic algorithm can be generalized to solve other routing problems, e.g., the capacitated vehicle routing problem.

1. Introduction

Vehicle routing problem (VRP) is a core problem in logistics. In recent years, a widely studied variant of the VRP is the team orienteering problem (TOP). In this problem, a fleet of vehicles are scheduled to serve a set of nodes (or customers), each of which is associated with a reward. Once a node is served by a vehicle, the corresponding reward will be received. The objective is to maximize the total received reward while the travel time of each route must be not more than a time limit. The TOP was first formulated for team orienteering, which is a kind of outdoor sport. It can be used to model a broad range of real-life applications such as tourist trip planning [1] and sportsman recruitment [2]. An interesting application is unmanned aerial vehicle (UAV) mission planning [3] in which some UAVs attached with various sensors are required to patrol a set of targets. Each target is assigned a profit, and the planner aims to maximize the total received profit.

The TOP is a multi-level combinatorial optimization problem [2]. The first level is to choose a subset of nodes to visit, the second level is to assign the chosen nodes to each vehicle, and the third level is to find a route through the nodes assigned to each vehicle. It is well known that the TOP is NP-hard [2]. So far several exact algorithms [4–7] have been developed. However, unless P=NP, there is no algorithm which can find an optimal solution within time polynomial in the size of customers. An alternative approach to the TOP is metaheuristic, which aims to yield a satisfactory solution within reasonable time. To date, ant colony optimization (ACO) [8], guided local search [9], large neighborhood search [10], memetic algorithm (MA) [11], particle swarm optimization [12], path relinking [13], tabu search [14,15], and variable neighborhood search [15] have been applied to the TOP. The interested reader is referred to [1] for a survey. Among the current metaheuristics, the particle swarm optimization algorithm in [12] ranked first when testing on the instances of [2]. Moreover, the results on larger new generated instances demonstrate that their algorithm is very effective though it consumes relatively long running time.

In the design of a metaheuristic [16], solution generation is critical to the performance. When generating new solutions, a metaheuristic employs specific operators by using one or more incumbent solutions. These operators can be categorized as constructive or non-constructive. For example, ACO [8] adopts a construction procedure which depends on pheromone trails and heuristic information. The limitation is that it is somewhat complex to maintain the pheromone trails. MA [11] uses a non-constructive operator, i.e., crossover to generate new individuals, each of which is represented by an ordered sequence of the customers. It decodes a sequence into a solution by a so-called split [17] or route first-cluster second approach [18]. To make a metaheuristic work better, it is challenging to investigate simple operators so as to exploit the incumbent solutions better.

☆This manuscript was processed by Associate Editor Yagiura.

* Corresponding author.
E-mail address: kelj163@163.com (L. Ke).

http://dx.doi.org/10.1016/j.omega.2015.08.003
0305-0483/

Solution update rule determines the criterion on choosing new incumbent solutions. As far as we know, the update rules of the current metaheuristics for the TOP are quality dependent. That is, new incumbent solutions are chosen based on their objective values. It is no doubt that one will expect that those chosen solutions are of high-quality and low-similarity, thereby leading to explore potential and wide search areas. It is therefore beneficial to highlight the similarity in the update rule. In the literature, multi-criteria decision has proven to be useful to preserve the diversity of the incumbent solutions [19,20]. Here we adopt this technique to deal with the TOP for the first time.

In this paper, a metaheuristic, called Pareto mimic algorithm, is proposed. The main contributions are summarized as follows: (1) It uses a new operator, called mimic operator, to generate a new solution by imitating an incumbent solution. Via a parameter, this operator easily controls the similarity between the generated solution and the incumbent solution. (2) It updates a population of incumbent solutions based on Pareto dominance. In fact, to determine, incumbent solutions can be modeled as a multi-criteria decision problem [21]. This prompts us to define multiple indicators to measure each candidate solution. Since a solution may be not better than others in terms of all indicators, Pareto dominance [21], a concept that was first used in economics, is adopted for updating incumbent solutions. Due to the mimic operator and Pareto dominance based update rule, the proposed algorithm is named after *Pareto mimic algorithm* (PMA). (3) It adopts a new operator, called swallow operator, to improve a solution. This operator attempts to swallow (or insert) an infeasible node and then repair the resulting infeasible solution. In contrast to local search, the swallow operator allows the search to tunnel through the infeasible solution area. The idea of tunneling through the infeasible solution area has also been used in [13,15].

The remainder of this paper is structured as follows. Section 2 presents a description of the TOP. Section 3 presents the proposed algorithm. Section 4 presents the experimental study. Section 5 discusses how to generalize PMA for other routing problems. Finally, Section 6 concludes the main results.

2. A description of the top

The TOP is defined on a complete graph $G=(V,E)$, where $V=\{0,1,\cdots,n+1\}$ is the set of nodes and $E=\{(i,j)|i,j\in V\}$ is the set of edges. Let c_{ij} be the travel time of edge $(i,j)\in E$ and r_i be the reward of node i. A feasible path must start at node 0 and finish at node $n+1$, and its travel time cannot exceed a time limit $T_{\max}$. Let Ω be the set of all feasible paths. There are m vehicles.

Let $R(x_k)$ be the total received reward of a path $x_k\in\Omega$. Symbol $a_{ik}=1$ if path $x_k\in\Omega$ visits customer i ($1\le i\le n$), otherwise $a_{ik}=0$. Variable $y_k=1$ if path $x_k\in\Omega$ is traveled, otherwise $y_k=0$. The TOP can be stated as follows [5]:

$$\begin{aligned}
&\text{maximize} && \sum_{x_k\in\Omega} R(x_k)y_k \\
&\text{subject to} && \sum_{x_k\in\Omega} a_{ik}y_k\le 1, \quad 1\le i\le n \\
& && \sum_{x_k\in\Omega} y_k\le m \\
& && y_k\in\{0,1\}, \quad x_k\in\Omega
\end{aligned} \tag{1}$$

In this formulation, the goal is to maximize the total received reward. The first constraint means that each node only can be visited at most once, and the second constraint ensures that there are at most m paths in a feasible solution.

3. The proposed algorithm

Pareto mimic algorithm, denoted as PMA, is an iterative solution technique. At first, N incumbent solutions are initialized. The best-so-far solution x_b is set as the one with the largest objective value among these N solutions where the objective value of a solution x, denoted by $F(x)$, is the total received reward of those paths of x. At each step, it works as follows: starting from each incumbent solution, a new solution is generated by the mimic operator. Subsequently, local search and the swallow operator are employed to improve it. By using the new generated solutions and those old incumbent solutions, the set of incumbent solutions, denoted as IS, is updated according to Pareto dominance. The best-so-far solution x_b is also renewed. PMA terminates when a stopping condition is satisfied. The main procedure of PMA is given in Algorithm 1.

Algorithm 1. The main procedure of the proposed algorithm.

Input: A TOP to be solved
Output:the best-so-far solution x_b
1: set $IS:=\varnothing$
2: set x_b as the solution which contains no node
3: **for** $i=1$ **to** N **do**
4: $x:=$ **Initialization**() /*see Section 3.2*/
5: set $IS:=\{x\}\bigcup IS$
6: replace x_b by x if $F(x)>F(x_b)$
7: **end for**
8: **while** the termination condition is not satisfied **do**
9: set $U:=$ the size of IS
10: set $Q:=\varnothing$/*Q is an auxiliary set*/
11: **for** $i=1$ **to** U **do**
12: $x:=$ **MimicOperator**(the ith solution of IS)/*see Section 3.3*/
13: $x:=$ **LocalSearch**(x) /*see Section 3.4*/
14: $x:=$**SwallowOperator**(x) /*see Section 3.5*/
15: set $Q:=\{x\}\bigcup Q$
16: replace x_b by x if $F(x)>F(x_b)$
17: **end for**
18: $IS:=$ **Update** ($Q\cup IS$)/*see Section 3.6*/
19: **end while**

3.1. Notations

(1) *Static preference value*: This value indicates the favorite degree of transiting from one node to another node. Suppose the current node is i, the static preference value of node j is defined as r_j/c_{ij}.

(2) *Dynamic preference value:* Given a solution x, let $\Delta T(x,i,k)$ be the increment of travel time caused by inserting i into the kth best position. The dynamic preference value of node i, denoted as $D(x,i)$, is defined as follows:

$$D(x,i)=(\Delta T(x,i,3)-\Delta T(x,i,1))\cdot r_i^u \tag{2}$$

where $u\in(0,1]$ is a uniformly distributed random number. $\Delta T(x,i,3)-\Delta T(x,i,1)$ is the difference of travel time obtained by inserting i into the third best and the best position. We also tested other possibilities, e.g., the second best position instead of the third best one in Eq. (2), but the presented choice gives us the best performance (see Section 4.4). In some references, this difference is also called regret value [22,23]. We adopt this definition of regret value since it is effective and cheap in computation. If there is only one position to be inserted, $\Delta T(x,i,3)$ is set as a sufficiently large value, which means that it is urgent to insert node i. A larger dynamic preference value indicates that the node is more favorable. In contrast to the static preference value, the dynamic preference value depends on the starting solution and it is randomized.

(3) *Feasible node:* Given a solution s, a node is said to be feasible if there exists at least one position to insert it such that the resulting solution satisfies the time limit constraint.

(4) *Removal reward:* Given a solution s, the removal reward of node i (i belongs to s) is defined as T_i/r_i where T_i is the decrement of travel time obtained by removing node i.

(5) *Distance between two solutions:* For any solution x, it is associated with an n-dimensional vector W, if node $i(1 \le i \le n)$ is visited, then $W_i=1$, otherwise, $W_i=0$. Given two solutions x and y, their distance, denoted by $d(x,y)$, is defined as the Hamming distance between their corresponding n-dimensional vectors W and V:

$$d(x,y) = Hamming\ Distance(W,V) \tag{3}$$

3.2. Initialization of incumbent solutions

The initialization procedure (**Initialization**) constructs m paths in the same manner: at each step, suppose that the current node is i, μ is the number of unvisited feasible nodes, $l=\min(\mu,\gamma)$ where γ is an integer parameter. If l is nonzero, then the next node is randomly chosen from the best l nodes in terms of their static preference values, otherwise one path is finished at the ending node. To speed up the procedure, one can apply a preprocessing procedure to store some favorite nodes for each node. In detail, for any node i ($1 \le i \le n$), the nodes except $0, i$, and $n+1$ are sorted in descending order according to their static preference values, and then the first L nodes are stored into a list of node i where L a sufficient large integer (in our experiment, L is chosen as 50). In this way, it is very fast to determine the best l nodes at each construction step.

3.3. The mimic operator

By using an incumbent solution x_I as the reference solution, the mimic operator (**MimicOperator**) uses the similarity ratio, denoted as α, to control the similarity between the new solution x and x_I. This operator builds a new solution by a similar procedure in Section 3.2. Their difference only lies in the use of x_I. Suppose the current node is i, the next node is determined as follows. First, a uniformly distributed random number $r \in [0,1]$ is generated. Let the successor of i in x_I be j. If $r<\alpha$ and j is feasible, the next node is set to j. Otherwise, the same procedure as the one in Section 3.2 is conducted. The main procedure is given in Algorithm 2. Lines 8–13 check the next node in x_I, which is the main difference between the mimic operator and the initialization procedure.

Algorithm 2. The mimic operator.

Input: An incumbent solution x_I
Onput: a new solution x
1: set *currentNode* : $=0$
2: **for** $i=1$ **to** m **do**
3: /*construct the ith path*/
4: set the ith path R_i:=∅
5: **while** 1 **do**
6: set *flag* := false
7: generate a uniformly distributed random number $r \in [0,1]$
8: **if** $r<\alpha$ **then**
9: set node := the next node of *currentNode* in x_I
10: **if** *node* is feasible **then**
11: set nextNode := node, flag:= true
12: **end if**
13: **end if**
14: **if** *flag*=false **then**
15: set μ:= the number of unvisited feasible nodes
16: $l=\min(\mu,\gamma)$
17: **if** $l>0$ **then**
18: choose a *node* randomly from the best l nodes in terms of their static preference values
19: set *nextNode*:=*node*, *flag*:= true
20: **end if**
21: **end if**
22: **if** *flag*=true **then**
23: add *nextNode* into the path R_i
24: set *currentNode*:=*nextNode*
25: **else**
26: **break**;
27: **end if**
28: **end while**
29: **end for**

It should be pointed out that the relinking operator in [13] is similar to the mimic operator. However, it intentionally generates infeasible solutions to explore the region of the search space between two solutions.

3.4. Local search

In our algorithm, local search (**LocalSearch**) consists of two steps. The first step aims to shorten the total travel time. The second step attempts to increase the total received reward. In the first step, 2-opt, exchange, cross and relocation operator are employed in turn. Once all these operators cannot find an improved solution, the first step stops and the second step begins. Otherwise, it continues. These operators use a move to generate a neighborhood. The description of these moves can be found in [24] or see Fig. 1. Each local search operator conducts as follows: at each step, the operator compares the current solution and the best neighboring solution which has the smallest travel time among the solutions in the neighborhood of the current solution. If the travel time of the best neighboring solution is smaller than the one of the current solution, then the best neighboring solution will be accepted as the new current solution and the operator repeats. Otherwise, the operator stops. Notice that these operators do not consider those unvisited nodes.

The second step uses the insertion and exchange-unvisited operator in the sequel. Once both of the operators used in the second step cannot find a better solution, local search stops. Otherwise, this step continues.

3.4.1. The insertion operator

This operator greedily inserts one node at a time. At each step, the feasible node with the largest dynamic preference value is chosen and inserted into the best position of the incumbent solution. This procedure is repeated until no feasible node can be inserted.

3.4.2. The exchange-unvisited operator

This operator works as follows: for each visited node, if there is an unvisited node which is feasible and can increase the total received reward, then they will be exchanged. This procedure is repeated until no better solution can be found.

3.5. The swallow operator

Starting from a solution x, the swallow operator (**SwallowOpeator**) tries to insert an unvisited node with the largest dynamic preference value (see Eq. (2)). Since the resulting solution is

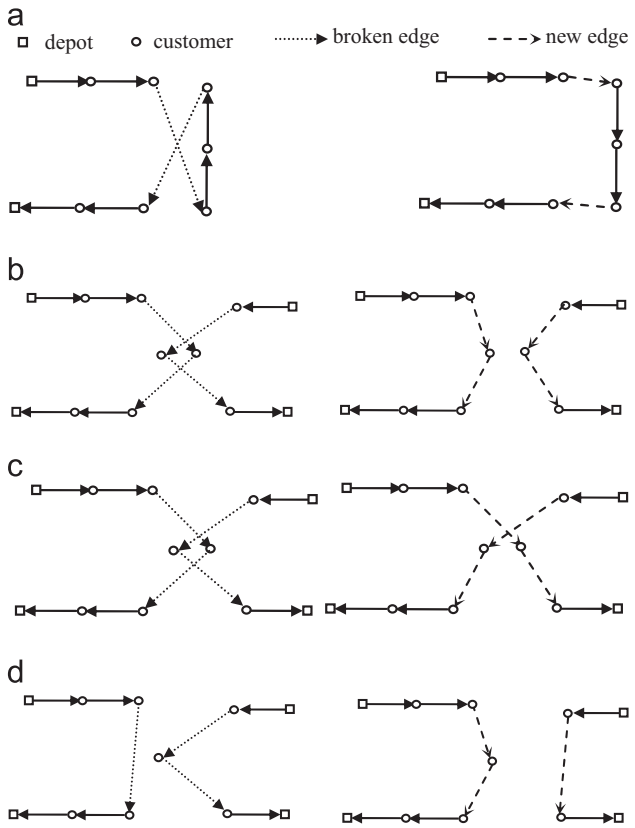

Fig. 1. The illustration of the four moves, each of which transforms the left into the right. (a) 2-opt move, (b) exchange move, (c) cross move, and (d) relocation move.

infeasible, a repair procedure is applied: suppose that node i is inserted into route R_l, local search is applied with a different acceptance condition: a neighboring solution is accepted if it is the first one that can shorten the travel time of route R_l and keep the feasibility of other routes. Once a local search operator repairs the solution (i.e., the solution becomes feasible), the repair procedure is stopped. Otherwise it will be further repaired by a removal procedure. The removal procedure removes the node with the largest removal reward step by step until the final solution is feasible. If the final solution is better than the starting solution x, then x is replaced and the swallow operator continues. Otherwise, the operator stops.

The swallow operator intentionally makes a feasible solution into an infeasible one by inserting an infeasible node and then repairs it. The idea of repair has been used in the literature. For example, in [25], a repair operator is used to modify an infeasible solution obtained by the solution generator which is possible to generate infeasible solutions. Nevertheless, the solution generator we use only generates feasible solutions and the swallow operator is merely applied on a feasible solution.

3.6. Pareto dominance based rule for Updating of IS

PMA assigns two indicators to each solution x: $F(x)$ and $N(x)+d(x,x_b)$ where $N(x)$ is the number of visited nodes, and x_b is the best-so-far solution. The first indicator corresponds to the solution quality. The second one considers the influence of the number of visited nodes and the distance away from x_b as well, which is used to preserve the diversity of the incumbent solutions. A solution x is said to *dominate* another solution y, denoted as $x \succ y$, if the following conditions hold: (1) x is not worse than y with respect to the two indicators. (2) x is strictly better than y in terms of at least one indicator. A solution is said to be *nondominated* if no other solution dominates it.

The update procedure (**Update**) chooses the new incumbent solutions from a set of candidate solutions consisting of the old solutions in *IS* and new generated solutions as follows: At first, those solutions which visit less than $N(x_b)-N$ nodes are removed since their objective values are rather poorer than x_b. Choosing such a threshold $N(x_b)-N$ is motivated by the following observation: A solution with smaller number of visited nodes may be poorer in terms of the objective value. The threshold is used such that the accepted solutions are not too poor. Secondly, the nondominated solutions are chosen from the remaining solutions. If the number of the nondominated solutions is larger than N, then N solutions are chosen according to *crowding-distance*. The crowding distance, proposed in [26], is an indicator which measures the density for every solution in a population. Given a nondominated solution s, its crowding distance, denoted by d_s, is defined as follows: If it is a boundary solution, then the largest value $(+\infty)$ is assigned to it. Otherwise, notice that the neighboring solutions along each indicator form a cuboid which contains solution s (see Fig. 2). The crowding distance is defined as the average side length of the cuboid. A smaller crowding distance means that the solution belongs to a more dense area of the space spanned by the two indicators, or vice versa.

3.7. Stopping condition

There are different stopping conditions adopted in the literature, e.g., the maximum running time limit and the maximum number of iterations. In this paper, the algorithm stops iterating either when the maximum running time have achieved $700\lceil(n+2)/100\rceil$ s (in this way, the maximum time limit of the maximum instance is less than 3600 s) or when no better solution could be found for 3000 iterations. In some sense, it is reasonable to claim that the algorithm has converged or consumed so much time.

4. Experimental study

PMA was coded in C++ and tested on a PC with Core i5, 3.2 GHz CPU, and 4 GB RAM. It has been tested on the instances in [2]. In these instances, the numbers of nodes are not more than 102. We observed that our algorithm can find the best known results for these instances except p4.4.n (our result is 976 while the best-known result is 977 [11]). So far, only the algorithms in [10,12] can achieve all of the best solutions found by our algorithm. Due to the sizes of these

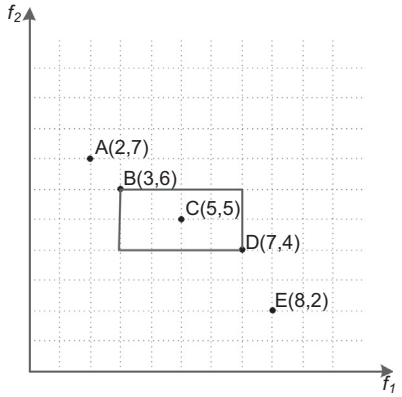

Fig. 2. An illustration of crowding distance. The crowding distance of Points *A*, *E*, and *C* is ∞, ∞ and 3 respectively. The neighboring points of *C* are *B* and *D* which form a cuboid shown in boldface rectangle. The average side length of the cuboid which contains *C* is (4+2)/2 (i.e., 3).

instances, the difference between these algorithms is tiny (see Appendix).

Recently, [12] realized that it is necessary to generate larger instances for performance assessment. 333 instances were generated by using the orienteering problem (OP) of [27] according to the transformation of [2]. This transformation sets the time limit of each vehicle of TOP (T_{max}) as $T_{max} = T^{OP}_{max}/m$ where m is the number of vehicles of the new TOP instance and T^{OP}_{max} is the time limit of the vehicle of the corresponding OP instance. Ref. [27] generated two classes of instances. The first class was transformed from instances of the Capacitated Vehicle Routing Problem (CVRP) [28,29] in which customer demands were transformed into rewards and different values of T^{OP}_{max} were considered. The second class was transformed from instances of the Traveling Salesman Problem (TSP) [29] in which customer rewards were generated in three different methods: (1) each reward is equal to 1 (gen1); (2) each reward is assigned to a value randomly sampled from [1, 100] (gen2); (3) each reward is assigned by a distance dependent function such that small rewards are assigned to nodes close to the depots (gen3).

Since only the results obtained by the algorithm in [12], called Particle swarm optimization-inspired algorithm (PSOiA), are available, we compared PMA with PSOiA in terms of the best and mean rewards and the average running time. As done in [12], PMA was tested on each instance 10 times.

4.1. Parameter setting

There are three parameters in PMA, that is, the maximum number of incumbent solutions N, the similarity ratio α and γ. The parameter values are determined using statistical study. The tested values of these parameters are as follows: $N \in \{1, 5, 10, 20, 50\}$, $\alpha \in \{0.5, 0.75, 0.85, 0.95, 0.99\}$, and $\gamma \in \{1, 5, 10, 20, 50\}$. Based on experimental results, we observed that when $N = 10$, $\alpha = 0.95$, $\gamma = 10$, PMA works best. To analyze the effect of each parameter, only one parameter is varied while others are fixed.

N determines how many good solutions are accepted for generating new solutions. When $N = 1$, only the best-so-far solution is accepted at each iteration. As shown in Table 1, the algorithms works the worst with respect to the best and mean value obtained on each tested instance. As N increases, the solution quality improves. But when $N = 20, 50$, the results become worse. This indicates that it may be not beneficial to accept too much inferior solutions.

α controls the similarity between an incumbent solution and the new generated solution. Table 2 reports the computational results obtained when different values of α are tested. When $\alpha = 0.5$, the best and mean value obtained on each instance are the worst. In this case, the construction procedure is the most diverse. This indicates that it is useful to imitate an incumbent solution for solution construction. When $\alpha = 0.95$, the performance is the best. The results also support the usefulness of the mimic operator. When $\alpha = 0.99$, the results become worse, which may be due to that the algorithm works overly greedy.

γ plays an important role in the mimic operator. With a smaller value, the positive effect is that those favorite nodes are more possible to be chosen, while the negative effect is that the algorithm may lack diversity. According to Table 3, one can notice that when $\gamma = 1$, the algorithm obtains the worst results. When $\gamma = 10$, the results are the best. As γ increases, the results become worse. It is therefore important to balance the intensity and diversity.

4.2. The effect of the Pareto dominance based update rule

An update rule determines the new population of solutions from the new generated solutions and the previous population. In PMA, the Pareto dominance based update rule is adopted. Three other update rules are also concerned. The first one is the rank-based rule in which the N best solutions are chosen. The second one is the roulette-wheel rule in which a solution x is randomly selected with a probability $F(x)/\Xi$ where $F(x)$ is the objective value of solution x and Ξ is the sum of the objective values of all candidate solutions. The third one is the paired-comparison rule which works as follows: let s_i be the i-th solution in IS, suppose that the new solution generated from it is sn_i, if $F(sn_i) > F(s_i)$, then s_i is replaced with sn_i. The comparison results are presented in Table 4. One can notice that the Pareto dominance based rule is better than the other three. This observation is in favor of the Pareto dominance based rule.

4.3. The use of the swallow operator

To study the effect of the swallow operator, we tested PMA without using the swallow operator on the same instances. As seen from Table 5, it is consistent that the swallow operator can help us to improve the performance on all these instances.

4.4. Comparison of the third best position and the second best position

In our implementation, the third best position is adopted in calculating regret value. The second best position may be a good choice. It is more aggressive than the third best position, however it is slightly cheaper in computing. As seen from Table 6, the third best position is better.

4.5. Comparison with the algorithm in [12]

In [12], the authors observed that the average relative percentage error (*ARPE*) between the best value (*best*) and the mean value (*mean*), defined by $(best - mean)/best \times 100\%$, is zero in 251 instances and they only reported the results of other 82 instances.

Table 7 reports the results obtained by PSOiA and PMA. Columns 1–5 present the information of each instance (i.e., name,

Table 1
Computational results obtained when different values of N are tested.

Instance			$N=1$		$N=5$		$N=10$		$N=20$		$N=50$	
Name	*n*	*m*	*Best*	*Mean*	*Best*	*Mean*	*Best*	*Mean*	*Best*	*Mean*	*Best*	*Mean*
cmt151c _ m2	150	2	1964	1955.0	1964	1957.9	1964	1962.0	1964	1959.0	1963	1952.2
cmt151c _ m3	150	3	1907	1899.3	1911	1903.3	1916	1909.6	1911	1903.9	1907	1900.7
cmt151c _ m4	150	4	1879	1872.6	1880	1869.0	1880	1877.6	1880	1871.9	1880	1872.2
gil262c _ m2	249	2	11,002	10,926.3	11,020	10,972.6	11,030	11,016.5	11,019	11,001.9	11,012	10,949.9
gil262c _ m3	249	3	10,730	10,622.2	10,748	106,28.0	10,757	10,715.2	10,755	10,615.4	10,748	10,651.5
gil262c _ m4	249	4	10,256	10,113.1	10,275	102,12.2	10,281	10,267.3	10,278	10,193.8	10,275	10,204.3

Table 2
Computational results obtained when different values of α are tested.

Instance			$\alpha=0.5$		$\alpha=0.75$		$\alpha=0.85$		$\alpha=0.95$		$\alpha=0.99$	
Name	*n*	*m*	*Best*	*Mean*	*Best*	*Mean*	*Best*	*Mean*	*Best*	*Mean*	*Best*	*Mean*
cmt151c _ m2	150	2	1962	1954.0	1964	1956.8	1963	1957.2	1964	1962.0	1964	1958.3
cmt151c _ m3	150	3	1911	1906.4	1914	1908.2	1912	1905.4	1916	1909.6	1910	1904.0
cmt151c _ m4	150	4	1880	1872.4	1880	1869.7	1879	1869.4	1880	1877.6	1878	1866.8
gil262c _ m2	249	2	10,985	10,774.0	11,017	10,986.5	11,019	10,940.8	11,030	11,016.5	11,019	10,942.3
gil262c _ m3	249	3	10,729	10,598.3	10,741	10,581.6	10,748	10,609.7	10,757	10,715.2	10,747	10,614.4
gil262c _ m4	249	4	10,273	10,161.1	10,273	10,135.8	10,280	10,203.0	10,281	10,267.3	10,278	10,174.3

Table 3
Computational results obtained when different values of γ are tested.

Instance			$\gamma=1$		$\gamma=5$		$\gamma=10$		$\gamma=20$		$\gamma=50$	
Name	*n*	*m*	*Best*	*Mean*	*Best*	*Mean*	*Best*	*Mean*	*Best*	*Mean*	*Best*	*Mean*
cmt151c _ m2	150	2	1964	1957.9	1964	1958.1	1964	1962.0	1964	1959.2	1964	1960.0
cmt151c _ m3	150	3	1913	1908.1	1912	1904.1	1916	1909.6	1912	1906.5	1911	1905.2
cmt151c _ m4	150	4	1880	1869.3	1880	1871.8	1880	1877.6	1879	1873.3	1880	1872.5
gil262c _ m2	249	2	11,030	10,986.8	11,020	10,977.2	11,030	11,016.5	11,023	10,987.9	11,019	10,957.9
gil262c _ m3	249	3	10,750	10,668.4	10,720	10,607.6	10,757	10,715.2	10,751	10,672.0	10,754	10,632.0
gil262c _ m4	249	4	10,279	10,222.7	10,279	10,188.3	10,281	10,267.3	10,280	10,223.2	10,278	10,204.2

Table 4
Computational results obtained when different update rules are used.

Instance			Pareto dominance		Rank-based		Roulette-wheel		Paired-comparison	
Name	*n*	*m*	*best*	*mean*	*best*	*mean*	*best*	*mean*	*best*	*mean*
cmt151c _ m2	150	2	1964	1962.0	1962	1958.9	1951	1944.5	1962	1960.7
cmt151c _ m3	150	3	1916	1909.6	1911	1904.2	1903	1891.6	1911	1909.2
cmt151c _ m4	150	4	1880	1877.6	1880	1872.5	1863	1852.1	1880	1875.9
gil262c _ m2	249	2	11,030	11,016.5	11,019	10,985.9	10,934	10,822.5	11,030	11,004.9
gil262c _ m3	249	3	10,757	10,715.2	10,749	10,677.1	10,466	10,403.6	10,754	10,713.9
gil262c _ m4	249	4	10,281	10,267.3	10,278	10,210.7	10,073	10,002.6	10,274	10,259.4

Table 5
The comparison between PMA using and without using the swallow operator.

Instance			Using		Without using	
Name	*n*	*m*	*Best*	*Mean*	*Best*	*Mean*
cmt151c _ m2	150	2	1964	1962.0	1959	1943.3
cmt151c _ m3	150	3	1916	1909.6	1904	1889.3
cmt151c _ m4	150	4	1880	1877.6	1879	1856.5
gil262c _ m2	249	2	11,030	11,016.5	10,922	10,785.0
gil262c _ m3	249	3	10,757	10,715.2	10,664	10,521.0
gil262c _ m4	249	4	10,281	10,267.3	10,227	10,088.5

the number of customer, the number of vehicles, the time limit, and instance generation method). For each instance, Columns 6–8 report the best (*best*) and mean (*mean*) value and the running time (*time*) of PSOiA respectively, and Columns 9–11 report these values of PMA. In terms of the *best* value, our algorithm can find 10 new better results, and it finds the same value for the other 72 instances. With respect to the *mean* value, PMA is better than PSOiA on 56 out of 82 instances. We also noticed that the average *ARPE* value of PMA is 0.42 %. Therefore, PMA is very competitive in solution quality. As for the average time over 82 instances, the value of the PSOiA is 11031.0 s, while PMA is 996.4 s. Although the two algorithms were tested on different hardware (PSOiA was tested on a PC with AMD 2.6 GHz CPU) and their stopping conditions are not the same, it can be concluded that PMA can quickly converge to a high quality solution.

Table 6
Computational results obtained when the second best position and the third best position are used for calculating regret value.

Instance			Third-best		Second-best	
Name	*n*	*m*	*Best*	*Mean*	*Best*	*Mean*
cmt151c _ m2	150	2	1964	1962.0	1964	1959.2
cmt151c _ m3	150	3	1916	1909.6	1913	1908.6
cmt151c _ m4	150	4	1880	1877.6	1880	1876.9
gil262c _ m2	249	2	11,030	11,016.5	11,020	11,001.0
gil262c _ m3	249	3	10,757	10,715.2	10,745	10,696.8
gil262c _ m4	249	4	10,281	10,267.3	10,277	10,264.2

5. Discussion

One may wonder whether PMA can be generalized to solve other routing problems. As a population based metaheuristic, it consists of four main procedures: population initialization, solution generation, solution improvement, and population update. In PMA, a mimic operator is used to generate a new solution from every individual in the population. The population is updated by a Pareto dominance based rule. The outline of generalized PMA is given in Algorithm 3.

Algorithm 3. The outline of generalized PMA.

Input: a problem to be minimized
Output: the best-so-far solution x_b

Table 7
The results obtained on the 82 instances in [12].

Instances					PSOiA			PMA		
Name	*n*	*m*	T_{max}	*gen*	*Best*	*Mean*	*time(s)*	*Best*	*Mean*	*time(s)*
cmt101c _ m3	100	3	126.33	gen2	1300	1299.0	111.1	1300	1299.0	42.0
cmt151b _ m3	150	3	116.67	gen2	1385	1373.8	754.0	1385	1374.5	169.9
cmt151c _ m2	150	2	262.50	gen2	1963	1962.0	1799.6	**1964**	1962.0	368.5
cmt151c _ m3	150	3	175.00	gen2	1916	1909.1	1376.2	1916	1909.2	441.5
cmt151c _ m4	150	4	131.25	gen2	1880	1875.6	881.1	1880	1877.6	826.5
cmt200b _ m2	199	2	191.00	gen2	2096	2088.2	4181.0	2096	2086.8	669.4
cmt200b _ m3	199	3	127.33	gen2	2019	2005.0	2711.7	2019	2009.4	1351.7
cmt200b _ m4	198	4	95.50	gen2	1894	1889.7	1515.2	1894	1891.6	974.5
cmt200c _ m2	199	2	286.50	gen2	2818	2810.1	7320.3	2818	2810.6	1048.3
cmt200c _ m3	199	3	191.00	gen2	2766	2751.2	4217.3	2766	2751.8	1200.0
cmt200c _ m4	199	4	143.25	gen2	2712	2700.6	3004.1	2712	2703.3	1411.4
eil101b _ m3	100	3	105.00	gen2	916	913.8	134.4	916	914.6	160.6
eil101c _ m2	100	2	236.00	gen2	1305	1304.8	452.8	1305	1304.8	250.5
eil101c _ m3	100	3	157.33	gen2	1251	1244.1	227.6	1251	1244.2	95.0
gil262a _ m2	241	2	297.50	gen2	4078	4056.4	5907.3	4078	4066.3	2100.0
gil262a _ m4	112	4	148.75	gen2	3175	3174.2	271.8	3175	3175.0	222.6
gil262b _ m2	249	2	594.50	gen2	8081	8061.1	7473.2	8081	8074.1	1267.8
gil262b _ m3	249	3	396.33	gen2	7585	7574.9	7276.8	7585	7566.6	1027.2
gil262b _ m4	241	4	297.25	gen2	6781	6742.0	4878.6	6781	6756.7	912.6
gil262c _ m2	249	2	892.00	gen2	11,030	11,020.0	27,500.9	11,030	11,016.5	1309.0
gil262c _ m3	249	3	594.67	gen2	10,757	10,714.6	14,553.8	10,757	10,715.2	1375.6
gil262c _ m4	249	4	446.00	gen2	10,281	10,259.4	8472.0	10,281	10,267.3	1997.0
bier127 _ gen1 _ m2	126	2	29,570.50	gen1	106	104.8	1153.9	106	105.0	673.5
bier127 _ gen1 _ m3	126	3	19,713.70	gen1	103	102.4	591.9	103	102.5	470.1
bier127 _ gen2 _ m2	126	2	29,570.50	gen2	5464	5446.8	1132.6	5464	5454.9	398.8
bier127 _ gen2 _ m3	126	3	19,713.70	gen2	5393	5376.2	648.1	5393	5386.9	359.3
bier127 _ gen2 _ m4	120	4	14,785.20	gen2	5122	5119.2	657.6	**5123**	5120.2	383.8
bier127 _ gen3 _ m2	126	2	29,570.50	gen3	2885	2884.3	1301.3	2885	2884.7	296.7
bier127 _ gen3 _ m3	126	3	19,713.70	gen3	2706	2703.8	711.7	2706	2705.2	509.6
bier127 _ gen3 _ m4	120	4	14,785.20	gen3	2402	2384.6	680.8	2402	2398.6	227.7
gil262 _ gen1 _ m3	210	3	396.33	gen1	101	100.9	1769.3	101	100.2	482.6
gil262 _ gen1 _ m4	102	4	297.25	gen1	78	77.1	155.8	78	77.0	123.5
gil262 _ gen2 _ m2	261	2	594.50	gen2	7498	7457.8	7356.7	7498	7458.4	742.1
gil262 _ gen2 _ m3	210	3	396.33	gen2	5615	5608.2	3304.6	5615	5604.9	1163.8
gil262 _ gen3 _ m2	261	2	594.50	gen3	7183	7182.8	9129.3	7183	7180.0	284.2
gil262 _ gen3 _ m4	102	4	297.25	gen3	2507	2499.8	276.4	2507	2500.1	308.8
gr229 _ gen1 _ m4	227	4	441.25	gen1	223	220.8	11,922.0	223	220.3	46.6
gr229 _ gen2 _ m3	228	3	588.33	gen2	11,566	11,551.3	14,197.2	11,566	11,557.8	1665.3
gr229 _ gen2 _ m4	227	4	441.25	gen2	11,355	11,255.5	18,799.5	11,355	11,328.1	2272.0
gr229 _ gen3 _ m3	228	3	588.33	gen3	8056	8051.6	14,090.1	8056	8052.2	1065.3
gr229 _ gen3 _ m4	227	4	441.25	gen3	7621	7600.0	11,399.7	**7651**	7610.5	781.5
kroA150 _ gen2 _ m2	149	2	6631.00	gen2	4335	4334.4	893.0	4335	4335.0	495.9
kroA150 _ gen3 _ m3	127	3	4420.67	gen3	2726	2719.6	538.0	2726	2725.2	597.2
kroA200 _ gen1 _ m4	132	4	3671.00	gen1	81	80.4	560.3	81	80.6	503.4
kroB200 _ gen1 _ m2	199	2	7359.50	gen1	111	110.4	2344.5	111	110.3	663.9
kroB200 _ gen2 _ m2	199	2	7359.50	gen2	6185	6182.2	3467.3	6185	6183.4	426.7
kroB200 _ gen2 _ m4	128	4	3679.75	gen2	4944	4942.2	640.7	4944	4942.4	582.4
kroB200 _ gen3 _ m2	199	2	7359.50	gen3	4765	4757.8	6306.6	4765	4762.4	513.9
kroB200 _ gen3 _ m3	157	3	4906.33	gen3	3028	3016.0	1713.9	3028	3022.4	741.5
lin318 _ gen1 _ m2	317	2	10,522.50	gen1	180	170.1	20,667.2	180	175.3	1168.3
lin318 _ gen1 _ m3	256	3	7015.00	gen1	149	148.6	9014.6	149	147.9	721.4
lin318 _ gen2 _ m2	317	2	10,522.50	gen2	9544	9533.8	23,804.8	9544	9537.5	1924.6
lin318 _ gen2 _ m3	256	3	7015.00	gen2	7786	7782.1	9773.6	**7807**	7769.6	1413.5

Table 7 (*continued*)

Instances					PSOiA			PMA		
Name	*n*	*m*	T_{max}	*gen*	*Best*	*Mean*	*time(s)*	*Best*	*Mean*	*time(s)*
lin318 _ gen3 _ m2	317	2	10,522.50	gen3	7936	7905.6	44029.0	7936	7923.3	1547.3
lin318 _ gen3 _ m4	154	4	5261.25	gen3	3797	3796.4	1446.3	3797	3795.5	970.7
pr136 _ gen1 _ m2	131	2	24,193.00	gen1	63	62.7	451.1	63	62.8	107.5
pr136 _ gen2 _ m2	131	2	24,193.00	gen2	3641	3631.8	601.3	**3646**	3637.4	281.6
pr264 _ gen1 _ m4	118	4	6142.00	gen1	107	106.6	503.1	107	106.7	289.8
pr264 _ gen2 _ m2	131	2	12,284.00	gen2	6635	6634.2	2048.2	6635	6632.0	719.0
pr264 _ gen2 _ m3	131	3	8189.33	gen2	6420	6410.7	938.4	6420	6417.0	859.0
pr264 _ gen2 _ m4	118	4	6142.00	gen2	5584	5564.5	590.8	5584	5565.1	663.2
pr264 _ gen3 _ m3	131	3	8189.33	gen3	2772	2770.0	1037.5	2772	2769.8	922.5
pr299 _ gen1 _ m2	251	2	12,048.00	gen1	139	138.5	4775.9	139	138.3	573.4
pr299 _ gen1 _ m3	162	3	8032.00	gen1	111	110.1	1303.7	111	109.2	506.0
pr299 _ gen1 _ m4	112	4	6024.00	gen1	84	83.6	383.5	84	83.6	340.3
pr299 _ gen2 _ m3	162	3	8032.00	gen2	6018	5966.7	1446.0	6018	5979.2	909.7
pr299 _ gen2 _ m4	112	4	6024.00	gen2	4457	4453.0	593.4	4457	4455.2	767.7
pr299 _ gen3 _ m2	251	2	12,048.00	gen3	5729	5728.6	11,872.5	5729	5709.8	1489.5
pr299 _ gen3 _ m3	162	3	8032.00	gen3	3655	3611.0	2705.8	3655	3611.6	1058.2
pr299 _ gen3 _ m4	112	4	6024.00	gen3	2268	2258.0	455.6	2268	2261.8	402.4
rat195 _ gen2 _ m2	190	2	581.00	gen2	5148	5145.6	2157.0	5148	5145.9	886.6
rat195 _ gen3 _ m3	122	3	387.33	gen3	2574	2571.2	721.8	2574	2569.9	369.5
ts225 _ gen2 _ m2	217	2	31,661.00	gen2	5859	5858.5	2998.4	5859	5859.0	759.6
rd400 _ gen2 _ m2	399	2	3820.50	gen2	12,993	12,787.5	77,049.2	**13,045**	12,873.0	3220.6
rd400 _ gen2 _ m3	399	3	2547.00	gen2	12,645	12,372.1	53,707.1	12,645	12,543.9	2852.7
rd400 _ gen2 _ m4	399	4	1910.25	gen2	12,032	11,953.5	42,001.6	12,032	11,969.7	3299.3
rd400 _ gen1 _ m2	399	2	3820.50	gen1	230	227.8	56,767.3	**232**	228.5	3066.6
rd400 _ gen1 _ m3	399	3	2547.00	gen1	222	221.7	62,476.1	**224**	221.3	2844.0
rd400 _ gen1 _ m4	399	4	1910.25	gen1	213	210.6	34,744.8	213	209.9	1985.4
rd400 _ gen3 _ m2	399	2	3820.50	gen3	12,428	12,274.1	96,178.7	**12,431**	12,312.2	2418.4
rd400 _ gen3 _ m3	399	3	2547.00	gen3	11,639	11,629.5	68,074.8	11639	11,549.8	3500.0
rd400 _ gen3 _ m4	399	4	1910.25	gen3	104,17	10,383.1	48462.8	**10,436**	10,345.4	3500.0
Average					4434.4	4415.9	11,031.0	4436.1	4421.3	996.4

The boldface results mean that better solutions are found by PMA.

1: set $IS := \varnothing$
2: set the objective value of x_b as infinity.
3: **for** $i=1$ **to** N **do**
4: $x :=$ **PopulationInitialization**()
5: set $IS := \{x\} \bigcup IS$
6: replace x_b by x if $T(x) < T(x_b)$ /*T is the objective function*/
7: **end for**
8: **while** the termination condition is not satisfied **do**
9: set $U :=$ the size of IS
10: set $Q := \varnothing$/*Q is an auxiliary set*/
11: **for** $i=1$ **to** U **do**
12: $x :=$ **SolutionGeneration**(the ith solution of IS)
13: $x :=$ **SolutionImprovement** (x)
14: set $Q := \{x\} \bigcup Q$
15: replace x_b by x if $T(x) < T(x_b)$
16: **end for**
17: $IS :=$ **PopulationUpdate** ($Q \cup IS$)
18: **end while**

In the following, we will generalize PMA to solve another routing problem, the capacitated vehicle routing problem (CVRP). In contrast to the TOP, the CVRP has a different structure in which each node should be visited. The PMA for the TOP cannot be directly applied to the CVRP. There are many possible ways to implement every algorithmic components. We will show that PMA, at least possibly, can work well.

5.1. Problem description of the CVRP

In the CVRP, n customers are distributed in a weighted graph $G=(V,E)$ where $V=\{0,1,\ldots,n\}$ and $E=\{(i,j)|\, i,j \in V\}$ is the set of edges fully connected those nodes. The central depot is node 0 and the customer at node i has a demand d_i and a service time st_i, and each edge (i, j) is associated with a travel time c_{ij}. A fleet of identical vehicles are required to serve these customers, and each customer must be served only once by exactly one vehicle. Each route of a vehicle must start from and end at the central depot. The objective of the CVRP is to find an optimal solution such that the total travel time is minimized while the total travel time and the total demand of each route are not larger than the pre-specified limits T_{max} and D_{max} respectively.

5.2. Population initialization

N solutions are initialized by the following construction procedure: Suppose that the current position is node i, every other node j is assigned a preference value $\xi_{ij}=1/(c_{ij}+st_j)$. The procedure only examines a candidate list of the L most favorite nodes. Let C be the set of feasible nodes in the candidate list (that is, those unvisited ones that can be added into the current partial solution without violating the travel time and capacity constraints). If C is empty, a feasible node is randomly chosen out of the candidate list, otherwise, the next node j is chosen randomly with a probability given as follows:

$$p_j = \frac{\xi_{ij}}{\sum_{l \in C} \xi_{il}}, \quad j \in C$$

Once no feasible node can be selected, the central depot is chosen and the construction of a route is finished. The construction procedure of a solution continues until no customer is left to be served.

5.3. Mimic operator

Based on an incumbent solution x_I, the mimic operator constructs a new solution in the similar manner as the solution initialization procedure. Suppose the current node is i, the next node is determined as follows. First, a uniformly distributed random number $r \in [0,1]$ is generated. Let the successor of i in x_I be j. If $r < \alpha$ and j is feasible, the next node is set to j. Otherwise, the next node is selected by the same method as in the solution initialization procedure.

Algorithm 4. The main procedure of solution improvement for the CVRP.

Input: a solution x
Output: a solution x^*
1: set $x^* := x$
2: generate a new solution y by applying local search on x
3: replace x with y if $T(y) < T(x)$
4: set $R :=$ the number of routes in x
5: **for** $l=1$ **to** R **do**
6: **for** $j=1$ **to** TT
7: set $y := x$
8: generate a solution $\tilde{y}$ from y by 4-opt-based operator on the lth route of y
9: generate a new solution z by 2-opt on the ith route of $\tilde{y}$
10: replace x with z if $T(z) < T(x)$
11: **end for**
12: **end for**
13: replace x^* with x if $T(x^*) < T(x)$

5.4. Solution improvement

The solution improvement procedure consists of local search and an inner-improvement procedure. The details are presented in Algorithm 4.

5.4.1. Local search

The local search invokes 2-opt, exchange, cross and relocation operator one by one, and it repeats until no operator can find an improved solution. Each local search operator generates a neighborhood by a move [24]. This local search operator performs as follows: at each step, the operator compares the current solution and the best neighboring solution. The current solution will be replaced by the best neighboring solution if it is inferior. The operator iterates with the new current solution and it stops once no better solution can be found.

5.4.2. Inner-improvement procedure

This procedure aims to shorten each route of an incumbent solution. It repeats performing the following two steps until the trial times reaches TT: the first step uses a 4-opt-based operator (or double bridge operator, see [30]) to perturb and the second step is 2-opt.

Since the solution improvement procedure is time-consuming, it is employed on each solution with a probability p_{si}.

5.5. Population update

For each solution x, the solution quality and similarity indicators are defined as follows: $T(x)$ and $\mathfrak{N}(x_b,x)$ where $T(x)$ is the objective function value of x and $\mathfrak{N}(x_b,x)$ is the number of edges shared by x_b and x where x_b is the best-so-far solution. Note that solution x is said to be dominated by solution y if and only if x is not better than y in terms of both indicators (i.e., $T(x) \geq T(y)$, $\mathfrak{N}(x_b,x) \geq \mathfrak{N}(x_b,y)$) and x is strictly poorer than y with respect to at least one indicator which implies $T(x) > T(y)$ or $\mathfrak{N}(x_b,x) > \mathfrak{N}(x_b,y)$.

The new population is chosen from the candidate solutions (that is, the old solutions in IS and new generated solutions). At

Table 8
Computational results obtained on the 14 instances in [28].

Instances						Best known	GRASP × ELS[a]		MPNS-GRASP[b]		PMA	
	n	D_{max}	T_{max}	st	m		*Best*	*Time* (s)	*Best*	*Time* (s)	*Best*	*Time* (s)
1	51	160	∞	0	5	524.61	524.61	0.06	524.61	0.60	524.61	2.07
2	76	140	∞	0	10	835.26	835.26	15.01	836.39	5.40	835.26	9.71
3	101	200	∞	0	8	826.14	826.14	3.19	826.14	6.60	826.14	12.31
4	151	200	∞	0	12	1028.42	1029.48	44.84	1032.24	19.80	1029.79	23.98
5	200	200	∞	0	17	1291.29	1294.09	11.85	1314.25	55.20	1301.88	46.54
6	51	160	200	10	6	555.43	555.43	0.14	555.43	0.60	555.43	1.16
7	76	140	160	10	11	909.68	909.68	5.18	909.68	6.60	909.68	5.59
8	101	200	230	10	9	865.94	865.94	0.32	865.94	19.80	865.94	5.00
9	151	200	200	10	14	1162.55	1162.55	6.58	1175.86	35.40	1165.13	17.86
10	200	200	200	10	18	1395.85	1401.11	125.57	1412.11	72.60	1405.21	26.53
11	121	200	∞	0	7	1042.11	1042.11	0.64	1042.11	4.80	1042.11	12.43
12	101	200	∞	0	10	819.56	819.56	0.17	821.12	7.80	819.56	5.33
13	121	200	720	50	11	1541.14	1545.43	9.25	1548.53	10.20	1545.67	15.93
14	101	200	1040	90	11	866.37	866.37	0.27	868.62	7.20	866.37	8.54
Average						976.03	976.98	15.93	980.93	18.04	978.06	13.78

[a] GRASP × ELS was tested on a PC with 2.8 GHz CPU.
[b] MPNS-GRASP was tested on a PC with 1.86 GHz CPU.

Table 9
Computational results obtained on the 20 instances in [31].

Instances						Best known	GRASP × ELS[a]		MPNS-GRASP[b]		PMA	
	n	$D_{\max}$	$T_{\max}$	st	m		*Best*	*Time* (s)	*best*	*Time* (s)	*best*	*Time* (s)
1	240	550	650	0	10	5627.54	5644.52	238.02	5715.19	24.6	5641.27	40.26
2	320	700	900	0	11	8444.5	8444.5	93.12	8490.15	29.4	8460.93	54.11
3	400	900	1200	0	10	11,036.2	11,036.2	384.94	11,144.4	84.6	11,037.4	136.06
4	480	1000	1600	0	10	13,624.5	13,624.5	349.82	13,752.2	102.6	13624.5	84.1
5	200	900	1800	0	5	6460.98	6460.98	62.76	6475.19	16.8	6460.98	18.68
6	280	900	1500	0	7	8412.8	8412.9	84.44	8492.28	19.8	8412.9	65.22
7	360	900	1300	0	9	10,181.8	10195.6	463.21	10,275.2	34.2	10,268.4	81.58
8	440	900	1200	0	11	11,643.9	11,643.9	298.05	11,918.2	87.6	11,865.3	119.4
9	255	1000	∞	0	14	583.39	586.18	78.62	589.94	18	596.26	38.51
10	323	1000	∞	0	16	741.56	744.36	437.87	749.15	38.4	745.97	62.61
11	399	1000	∞	0	18	918.45	922.4	272.82	935.23	46.2	928.19	121.36
12	483	1000	∞	0	19	1107.19	1116.12	1746.33	1137.17	113.4	1125.86	102.49
13	252	1000	∞	0	26	859.11	860.55	128.69	875.14	45.6	862.75	28.59
14	320	1000	∞	0	30	1081.31	1084.82	546.93	1098.95	32.4	1095.87	61.96
15	396	1000	∞	0	33	1345.23	1352.39	613.36	1369.16	103.8	1363.26	65.47
16	480	1000	∞	0	37	1622.69	1634.27	1262.99	1651.14	159.6	1650.42	97.3
17	240	200	∞	0	22	707.79	707.79	61.63	715.16	35.4	710.12	30.29
18	300	200	∞	0	27	997.52	1002.15	175.55	1013.17	36	1029.07	28.83
19	360	200	∞	0	33	1366.86	1371.67	1078.54	1384.18	43.2	1386.37	41.99
20	420	200	∞	0	38	1820.09	1830.98	346.49	1835.18	72.6	1867.45	65.61
average						4429.17	4433.84	436.21	4480.82	57.21	4456.66	67.22

[a] GRASP × ELS was tested on a PC with 2.8 GHz CPU.
[b] MPNS-GRASP was tested on a PC with 1.86 GHz CPU.

first, those poor solutions are removed. In our implementation, a solution is considered to be poor if its objective value is larger than $1.02T(x_b)$. Second, the nondominated solutions are chosen from the remaining solutions. If the number of the nondominated solutions is larger than N, then N solutions are chosen according to crowding distance.

5.6. Experimental study

The algorithm was tested on two sets of benchmark problems: the 14 instances in [28] and the 20 large-scale instances in [31]. The parameters are set as follows: $N=20, L=40, \alpha=0.95, p_{si}=0.2, TT=10$. The algorithm stops when the maximum number of iterations reaches 1000. There are a huge number of algorithms for the CVRP (see [32–34] and references therein). In this study, we compare PMA with two algorithms in the literature. The one is GRASP × evolutionary local search hybrid (GRASP × ELS) in [33]. It should be highlighted that after GRASP × ELS was proposed, no new better solution has been reported. The other is multiple phase neighborhood search-GRASP (MPNS-GRASP) [34] which is the newest algorithm for the CVRP. Each instance was tested 10 times as done in [33,34].

Tables 8 and 9 present respectively the results for the 14 instances in [28] and the 20 instances in [31] obtained by the compared algorithms. Columns 1–6 present the information of each instance (i.e., name, the number of customer, the number of vehicles, the time limit, the capacity limit, service time of each node (*st*), and the number of vehicles). Column 7 presents the best known results [34]. For each instance, the best (*best*) and the running time (*time*) of each algorithm are reported. As seen from the results, PMA is slightly better than MPNS-GRASP. GRASP × ELS is the best, and on the two sets of benchmark instances, the average gaps between PMA and GRASP × ELS are 0.08% and 0.76% respectively.

Table 10
Computational results obtained on eight data sets.[a]

Set	PSOiA			PMA		
	Best	*Mean*	*Time* (s)	*Best*	*Mean*	*Time* (s)
*set*1	126.0	126.0	2.2	126.0	125.9	6.8
*set*2	140.5	140.5	0.4	140.5	140.5	1.4
*set*3	414.7	414.7	3.2	414.7	414.7	9.6
*set*4	862.1	860.9	218.6	862.1	859.3	109.3
*set*5	797.9	797.7	49.5	797.9	797.2	22.9
*set*6	788.5	788.5	47.1	788.5	787.9	36.4
*set*7	588.2	588.1	97.5	588.2	587.7	54.6
Remaining	1860.2	1860.2	2199.9	1860.2	1859.9	130.8

[a] The results are rounded off to the first decimal position.

Nevertheless, on these two sets, the average gaps between PMA and the best known results are 0.15% and 1.22% respectively. Therefore PMA is worse than the best 10 metaheuristics for the CVRP (see [32]).

6. Conclusion

This paper has introduced PMA to deal with the TOP. Our algorithm follows the general framework of the population-based metaheuristic [16] while it has three prominent features: (1) *The solution construction*: PMA employs the mimic operator to generate new solutions, which makes the algorithm easily inherit the information of high quality solutions. (2) *The solution update rule*: In our algorithm, a Pareto dominance based rule is adopted to update the incumbent solutions. This rule highlights the role of diversity and quality at the mean time. (3) *The solution improvement*: Especially the swallow operator is used to improve the performance. This operator can search in the infeasible region and act as an important companion of local search. Moreover, our algorithm emphasizes the use of the problem-specific information, such as static and dynamic preference value. According to the comparison study, PMA can achieve very promising performance.

We also extended PMA to solve the CVRP. The experimental results show that PMA is a potential metaheuristic for a wide range of routing problems. The study supports that the effectiveness of the proposed solution generating method and population update rule. It should be pointed out that PMA is by no means exclusive. In fact, it is of interest to combine other search mechanisms (e.g., ant colony optimization [8] and particle swarm optimization [12]) and diversity preservation strategies [35].

With regard to future work, it may be interesting to generalize PMA to solve other variants of the VRP, e.g., the routing problems in [36–38].

Acknowledgments

This work was supported by National Natural Science Foundation of China (No. 71471158, 61573277, and 61175063), the Research Grants Council of the Hong Kong Special Administrative Region, China (Project No. PolyU 15201414), State Key Laboratory of Intelligent Control and Decision of Complex Systems and the Fundamental Research Funds for the Central Universities, the Open Research Fund of the State Key Laboratory of Astronautic Dynamics under Grant 2014ADL-DW402 and 2015ADL-DW403, and the Scientific Research Foundation for the Returned Overseas Chinese Scholars, State Education Ministry. The authors also would like to thank The Hong Kong Polytechnic University Research Committee for financial and technical support. We are also thankful to the anonymous referees.

Appendix A

Table 10 reports the results obtained on the seven data sets in [2] (i.e., set1, set2,…, set7) and the data set, denoted by 'Remaining', consisting of 251 instances the details of which are only reported in the website of [12]. More details can be downloaded from http://keljxjtu.gr.xjtu.edu.cn

According to Table 10, our algorithm can find all the best known solutions. It is interesting that PMA consumes more time to solve set1, set2 and set3. This may be due to the stopping condition of PMA.

References

[1] Vansteenwegen P, Souffriau W, van Oudheusden D. The orienteering problem: a survey. European Journal of Operational Research 2011;209:1–10.
[2] Chao I-M, Golden BL, Wasil EA. The team orienteering problem. European Journal of Operational Research 1996;88:464–74.
[3] Kress M, Royset JO. Aerial search optimization model (ASOM) for UAVs in special operations. Military Operations Research 2008;13:23–33.
[4] Butt SE, Ryan DM. An optimal solution procedure for the multiple tour maximum collection problem using column generation. Computers & Operations Research 1999;26:427–41.
[5] Boussier S, Feillet D, Gendreau M. An exact algorithm for team orienteering problems. 4OR: A Quarterly Journal of Operations Research 2007;5:211–30.
[6] Dang D-C, El-Hajj R, Moukrim A. A branch-and-cut algorithm for solving the team orienteering problem. In: Lecture notes in computer science, vol. 7874; 2013. p. 332–9.
[7] Poggi M, Viana H, Uchoa E. The team orienteering problem: Formulations and branch-cut and price. In: The 10th workshop on algorithmic approaches for transportation modelling, optimization, and systems, vol. 14; 2010. p. 142–55.
[8] Ke L, Archetti C, Feng Z. Ants can solve the team orienteering problem. Computers & Industrial Engineering 2008;54:648–65.
[9] Vansteenwegen P, Souffriau W, Vanden Berghe G, van Oudheusden D. A guided local search metaheuristic for the team orienteering problem. European Journal of Operational Research 2009;196:118–27.
[10] Kim B-I, Li H, Johnson AL. An augmented large neighborhood search method for solving the team orienteering problem. Expert Systems with Applications 2013;40:3065–72.
[11] Bouly H, Dang D-C, Moukrim A. A memetic algorithm for the team orienteering problem. 4OR: A Quarterly Journal of Operations Research 2010;8:49–70.
[12] Dang D-C, Guibadj RN, Moukrim A. An effective PSO-inspired algorithm for the team orienteering problem. European Journal of Operational Research 2013;229:332–44.
[13] Souffriau W, Vansteenwegen P, Vanden Berghe G, van Oudheusden D. A path relinking approach for the team orienteering problem. Computers & Operations Research 2010;37:1853–9.
[14] Tang H, Miller-Hooks E. A tabu search heuristic for the team orienteering problem. Computers & Operations Research 2005;32:1379–407.
[15] Archetti C, Hertz A, Speranza MG. Metaheuristics for the team orienteering problem. Journal of Heuristics 2007;13:49–76.
[16] Talbi E-G. Metaheuristics: from design to implementation. John Wiley & Sons, New Jersey; 2009.
[17] Duhamel C, Lacomme P, Prodhon C. Efficient frameworks for greedy split and new depth first search split procedures for routing problems. Computers & Operations Research 2011;38:723–39.
[18] Beasley JE. Route first-cluster second methods for vehicle routing. Omega 1983;11:403–8.
[19] Neumann F, Wegener I. Minimum spanning trees made easier via multi-objective optimization. Natural Computing 2006;5:305–19.
[20] Segura C, Coello Coello CA, Miranda G, Léon C. Using multi-objective evolutionary algorithms for single-objective optimization. 4OR 2013; 11, 201–28.
[21] Ehrgott M. Multicriteria optimization. Springer, New York; 2005.
[22] Potvin J, Rousseau J. A parallel route building algorithm for the vehicle routing and scheduling problem with time windows. European Journal of Operational Research 1993;66:331–40.
[23] Ropke S, Pisinger D. An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows. Transportation Science 2006;40:455–72.
[24] Braysy O, Gendreau M. Vehicle routing problem with time windows, Part I: Route construction and local search algorithms. Transportation Science 2005;39:104–18.
[25] Gendreau M, Hertz A, Laporte G. A tabu search heuristic for the vehicle routing problem. Management Science 1994;40:1276–90.
[26] Deb K, Agrawal S, Pratap A, Meyarivan T. A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Transactions on Evolutionary Computation 2002;6:182–97.
[27] Fischetti M, Gonzalez JJS, Toth P. Solving the orienteering problem through branch-and-cut. INFORMS Journal on Computing 1998;10:133–48.
[28] Christofides N, Mingozzi A, Toth P. The vehicle routing problem. In: Combinatorial optimization. Wiley, New Jersey; 1979. p. 315–38.

[29] Reinelt G. Tspliba traveling salesman problem library. ORSA Journal on Computing 1991;3:376–84.
[30] Ke L, Feng Z. A two-phase metaheuristic for the cumulative capacitated vehicle routing problem. Computers & Operations Research 2013;40:633–8.
[31] Golden BL, Wasil EA, Kelly JP, Chao I-M. Metaheuristics in vehicle routing. In: Fleet management and logistics. Springer-Verlag, New York; 1998. p. 33–56.
[32] Laporte G. Fifty years of vehicle routing. Transportation Science 2009;43:408–16.
[33] Prins C. A GRASP × evolutionary local search hybrid for the vehicle routing problem. In: Bio-inspired algorithms for the vehicle routing problem. Springer, New York; 2009. p. 35–53.
[34] Marinakis Y. Multiple phase neighborhood search-GRASP for the capacitated vehicle routing problem. Expert Systems with Applications 2012;39:6807–15.
[35] Vidal T, Crainic TG, Gendreau M, Prins C. A hybrid genetic algorithm with adaptive diversity management for a large class of vehicle routing problems with time-windows. Computers & Operations Research 2013;40:475–89.
[36] Ma H, Cheang B, Lim A, Zhang L, Zhu Y. An investigation into the vehicle routing problem with time windows and link capacity constraints. Omega 2012;40:336–47.
[37] Ramos T, Gomes M, Barbosa-Póvoa A. Planning a sustainable reverse logistics system: balancing costs with environmental and social concerns. Omega 2014;48:60–74.
[38] Huang S-H, Lin P-C. Vehicle routing-scheduling for municipal waste collection system under the keep trash off the groundpolicy. Omega 2015;55:24–37.